Predicting Hourly Bike Rental Demand in Seoul Using Supervised Classification

Introduction to Data Science (INTDSCI), Final Project
De La Salle Lipa, College of Information Technology and Engineering
Computer Science Department, Section C2B
1st Semester, AY 2026–2027
The Good The Bad The Lodor: Artvincent Abesamis, Ian Lodor, Kurt Tengonciang
Instructor: Ms. Donna

A. Introduction

Seoul's public bike-sharing system has to keep bikes where people need them: busy stations run out during rush hour, while bikes sit unused when it rains or freezes. This project builds a supervised machine learning model that predicts whether a given hour will have **High Demand (505 or more rentals)** or **Low Demand (fewer than 505)** from the weather, the time of day and the calendar, so operators could move bikes before a rush instead of after docks are already empty. We trained and compared three models on one year of hourly data, and the output is a trained classifier, evaluated with a confusion matrix, that correctly labels 95.6% of hours it has never seen.

B. Data Collection and Preprocessing

B.1 Data source

The data is the Seoul Bike Sharing Demand dataset, collected from the Seoul Open Data Plaza and archived in the UCI Machine Learning Repository (Sathishkumar & Cho, 2020). It has **8,760 hourly records** (365 days x 24 hours) from **1 December 2017 to 30 November 2018**, with **14 columns**. Each row is one hour of the Seoul public bike-sharing system, with the number of bikes rented and the weather and calendar conditions for that hour.

The CSV contains the degree symbol (°) in two headers, so it is read with `encoding="latin1"` (Notebook cell 3).

B.2 Data dictionary

Variable	Original header	Type	Range or values	Description
Date	Date	datetime	01/12/2017 to 30/11/2018	Day of observation (DD/MM/YYYY)
Rented_Bike_Count	Rented Bike Count	integer	0 to 3,556	Bikes rented in that hour (raw target)
Hour	Hour	integer	0 to 23	Hour of day
Temperature_C	Temperature(°C)	float	-17.8 to 39.4	Air temperature

Humidity_Pct	Humidity(%)	integer	0 to 98	Relative humidity
Wind_Speed_ms	Wind speed (m/s)	float	0.0 to 7.4	Wind speed
Visibility_10m	Visibility (10m)	integer	27 to 2,000	Visibility in units of 10 m
Dew_Point_C	Dew point temperature(°C)	float	-30.6 to 27.2	Dew point
Solar_Radiation_MJm2	Solar Radiation (MJ/m2)	float	0.00 to 3.52	Solar radiation
Rainfall_mm	Rainfall(mm)	float	0.0 to 35.0	Rain in that hour
Snowfall_cm	Snowfall (cm)	float	0.0 to 8.8	Snow in that hour
Seasons	Seasons	category	Winter, Spring, Summer, Autumn	Season
Holiday	Holiday	category	Holiday, No Holiday	Public holiday flag (432 holiday hours)
Functioning_Day	Functioning Day	category	Yes, No	Whether the system was operating (295 hours are "No")

Two columns are derived later (Section C): Demand_Class (the target) and the calendar features Month and DayOfWeek.

B.3 Cleaning log (before and after)

Check	Before	Action	After	Why
Column names	Spaces, brackets and ° in 10 headers	Renamed to snake_case (Temperature(°C) to Temperature_C)	14 clean names	Safe to use in code (cell 6)
Date type	str	pd.to_datetime(format="%d/%m/%Y")	datetime64	Lets us extract month and weekday (cells 5 and 7)
Missing values	df.isna().sum() = 0	None needed	0	Nothing to impute (cell 6)
Duplicate rows	df.duplicated().sum() = 0	None needed	0	Nothing to drop (cell 6)
Row count	8,760	No rows removed	8,760	Nothing invalid was found
Closed hours (Functioning_Day = "No")	295 hours, all with 0 rentals	Kept	295	The system really was closed, and the model should learn that closed means Low demand

The data was already clean, so cleaning is limited to renaming and type conversion. No rows were deleted and no values were changed.

B.4 Exploratory data analysis

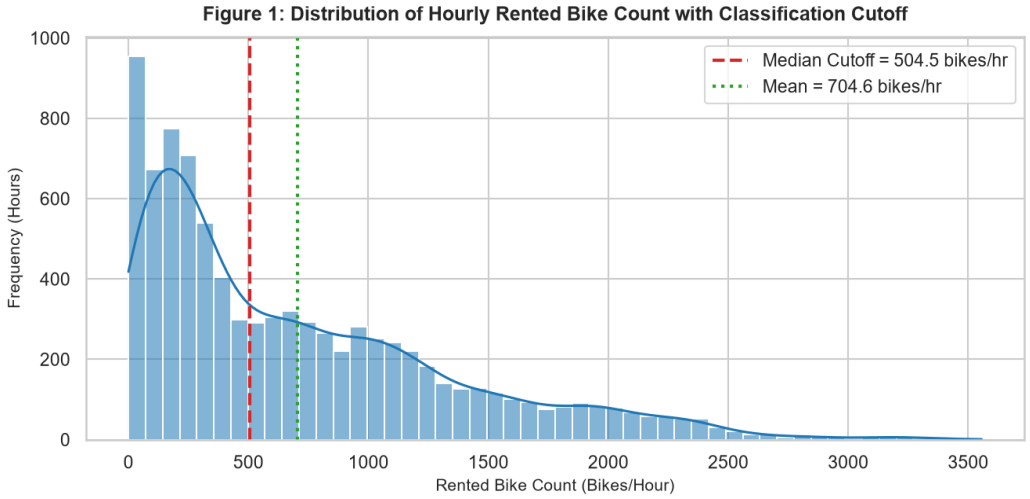


Figure 1. Distribution of hourly rentals with the classification cutoff (Notebook cell 9)

Hourly rentals are right-skewed (skewness 1.15). The mean is 704.6 and the median is 504.5, and the highest hour reached 3,556. We use the median as the cutoff, so hours with 505 or more rentals are High and the rest are Low. That gives exactly **4,380 Low and 4,380 High** hours, so accuracy is meaningful and a model cannot score well by always guessing one class.

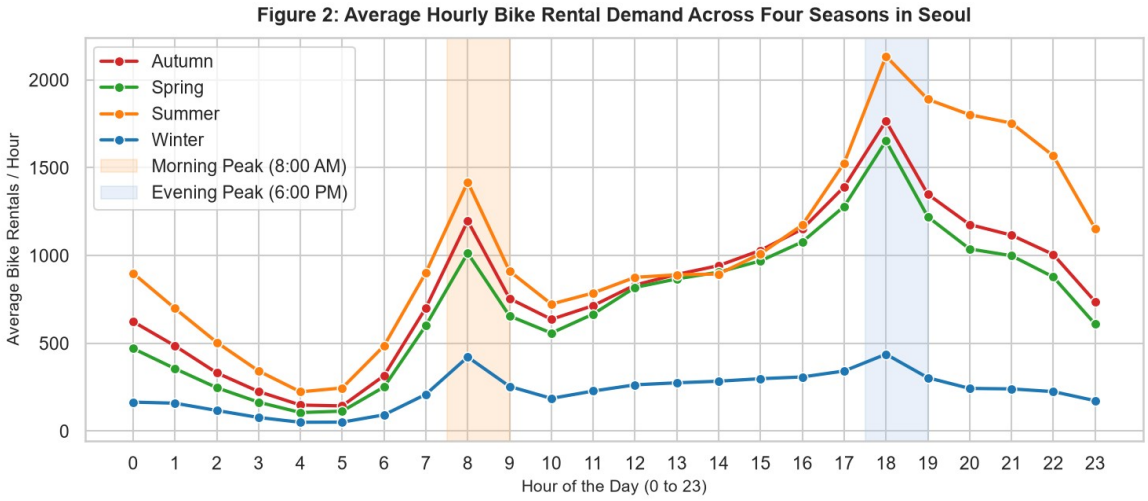


Figure 2. Average hourly rentals by season (Notebook cell 10)

Demand follows a commuter pattern: an 8 AM peak (all-season mean about 1,016 bikes) and a larger 6 PM peak (about 1,503). In Spring, Summer and Autumn the 8 AM average is 1,014 to 1,419 and the 6 PM average is 1,653 to 2,135. In Winter (mean temperature -2.5 °C) demand stays flat and low, and no hour of the day averages above 438 rentals. Winter is the clearest case where the hour of day only matters together with the weather.

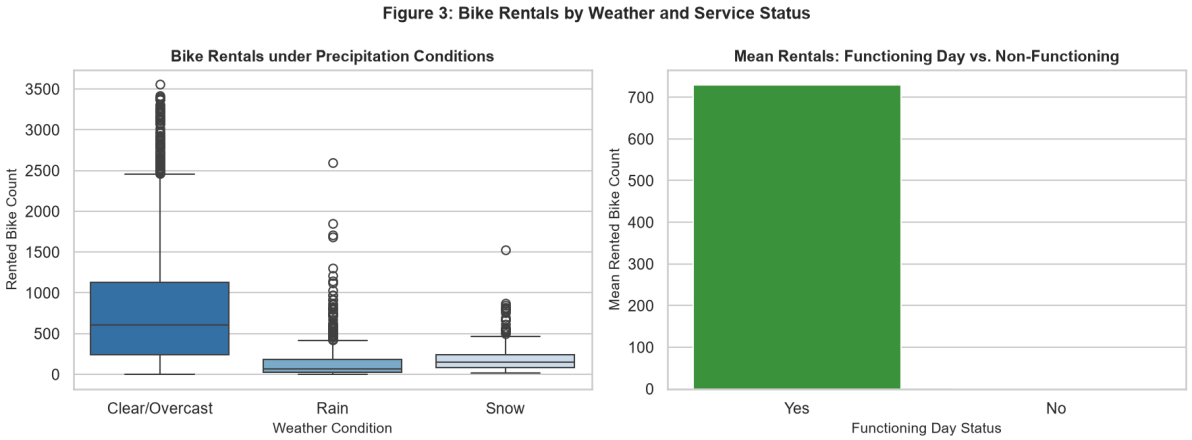


Figure 3. Rentals by weather and service status (Notebook cell 11)

The median is **606** rentals in clear hours (7,817 hours), **151** in snow-only hours (415 hours) and **67** in rain hours (528 hours). In the 295 hours when the system was not functioning, rentals are exactly 0.

Together the three figures show that hour, temperature, precipitation and service status carry most of the signal.

C. Feature Engineering

C.1 Target and leakage control

Demand_Class = 1 if Rented_Bike_Count >= 505, else 0 (cell 14). Keeping Rented_Bike_Count in the features would let the model read the answer, so it is dropped from X together with the raw Date. X has **14 columns**.

C.2 Calendar features

Month (1 to 12) and DayOfWeek (0 = Monday to 6 = Sunday) are extracted from Date (cell 14). A raw date string cannot be used by the models, but these two numbers capture the yearly and weekly cycles.

C.3 Correlation and feature selection

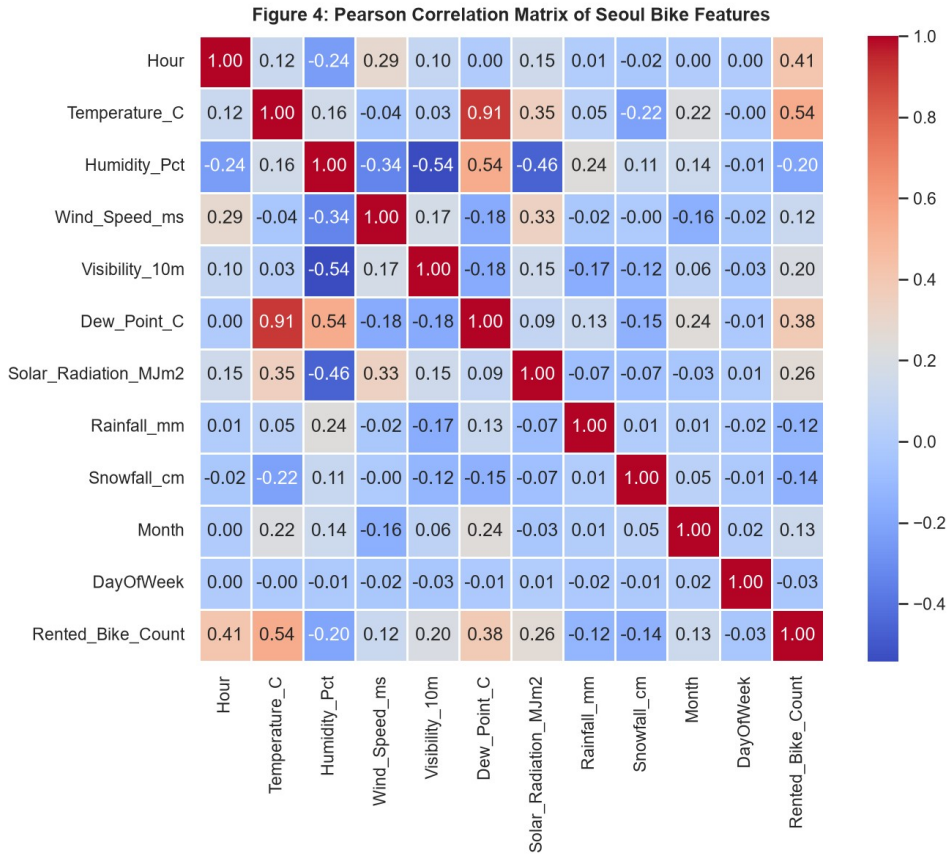


Figure 4. Pearson correlation matrix (Notebook cell 16)

Correlation with raw rental count:

Feature	r
Temperature_C	+0.54
Hour	+0.41
Dew_Point_C	+0.38
Solar_Radiation_MJm2	+0.26
Visibility_10m	+0.20
Month	+0.13
Wind_Speed_ms	+0.12
DayOfWeek	-0.03
Rainfall_mm	-0.12
Snowfall_cm	-0.14
Humidity_Pct	-0.20

Warmer, later-in-the-day and sunnier hours go with more rentals. Humidity, snow and rain go with fewer. DayOfWeek is close to zero but is kept because the models can still use it.

C.4 Multicollinearity check

Temperature_C and Dew_Point_C have r = **0.913**, so they carry almost the same information. We compared Logistic Regression with and without Dew_Point_C (cell 23): accuracy was **86.93%** with it and **86.99%** without it. That is a difference of one test hour out of 1,752, so dropping the column does not change the result. We keep all features in the main models and report the ablation as a check.

C.5 Preprocessing pipeline

Preprocessing is a ColumnTransformer fitted on the training set only (cell 21):

- **Numeric (11 columns):** StandardScaler (mean 0, standard deviation 1): Hour, Temperature_C, Humidity_Pct, Wind_Speed_ms, Visibility_10m, Dew_Point_C, Solar_Radiation_MJm2, Rainfall_mm, Snowfall_cm, Month, DayOfWeek.
- **Categorical (3 columns):** OneHotEncoder(drop="first"). Seasons becomes 3 columns (Spring, Summer, Winter; Autumn is the reference). Holiday becomes 1 column (Holiday_No Holiday; the reference is Holiday). Functioning_Day becomes 1 column (Functioning_Day_Yes; the reference is No).

The 14 input features become **16 numeric predictors** (11 + 3 + 1 + 1). Scaling and encoding are learned from the 7,008 training hours and then applied unchanged to the 1,752 test hours.

D. Data Splitting

We split the data into **80% training (7,008 hours)** and **20% testing (1,752 hours)** using `train_test_split` with `test_size=0.20` and `random_state=42`, as required. Fixing the random state means anyone who runs the notebook gets the same split and the same results.

Set	Hours	Low (0)	High (1)
Training	7,008	3,495	3,513
Testing	1,752	885	867

The scaler and encoder were **fit on the training set only** and then applied to the test set. This keeps the test set completely unseen, so the test score is a fair estimate of how the model handles new data. Each model is wrapped in a scikit-learn Pipeline together with its preprocessing, which guarantees this order every time it is trained.

E. Model Selection and Training

We trained three models on the same training set (notebook: Training cell).

Model	Settings	Why we used it

Baseline (DummyClassifier)	Always predicts the most common training class	Sets the minimum a real model must beat
Logistic Regression	max_iter=1000	The classification model from our lessons. Simple, fast, and easy to explain: it weighs each feature and turns the total into a probability of High Demand
Random Forest	100 trees, max depth 16, random_state=42	Our EDA showed demand jumps at 8 AM and 6 PM and drops in rain and winter. Those jumps aren't straight-line patterns, and decision trees can split on them directly. Combining 100 trees reduces the risk of one tree memorizing the training data

We started with logistic regression as our main model, as taught in class, and added Random Forest to test whether a model that handles non-linear patterns would do better on this data.

Experiments

Besides the three main models, we changed parts of the setup to see whether the results would move (notebook: Training, Limitation check and Closed-hours experiment cells).

Experiment	What we changed	Result	Decision
Remove dew point	Logistic regression without Dew_Point_C	86.93% to 86.99% (one test hour)	Kept all features
Remove closed hours	Dropped the 295 non-functioning hours	LR 86.83%, RF 94.86% (under 1 point change)	Kept the hours
Time-ordered split	Trained on the first 80% of the year, tested on the last 20%	RF 95.55% to 85.10%; LR 86.93% to 86.07%	Reported as a limitation (F.6)

F. Model Evaluation

F.1 Confusion matrices

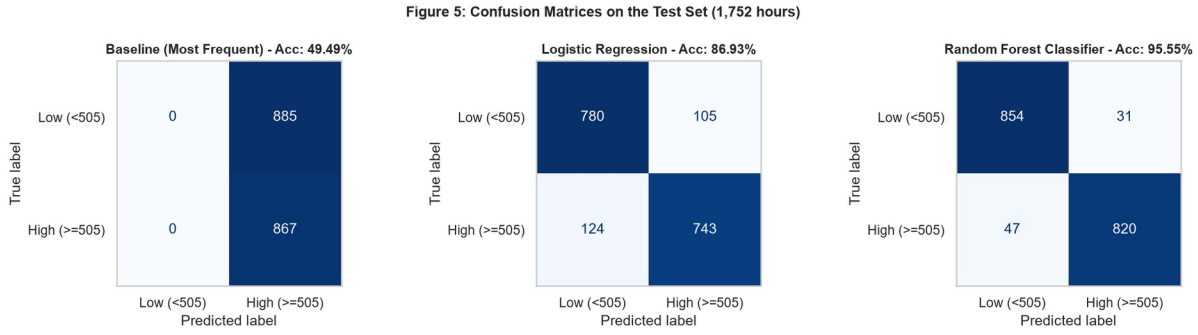


Figure 5. Confusion matrices for the three models on the 1,752 test hours (notebook: Confusion matrix cell)

Model	TN (Low, correct)	FP (false alarm)	FN (missed rush)	TP (High, correct)
Baseline	0	885	0	867
Logistic Regression	780	105	124	743
Random Forest	854	31	47	820

The baseline calls every hour High, so it gets every High hour right and every Low hour wrong. Logistic regression gets 1,523 hours right but makes 229 errors, slightly more misses (124) than false alarms (105). Random Forest cuts the errors to 78, about 66% fewer than logistic regression: it raises only 31 false alarms and misses only 47 busy hours.

F.2 Metrics

Model	Accuracy	Precision	Recall	F1-score
Baseline	49.5%	0.495	1.000	0.662
Logistic Regression	86.9%	0.876	0.857	0.867
Random Forest	95.6%	0.964	0.946	0.955

Accuracy is the share of all hours labelled correctly. Precision is how often a High prediction is really High. Recall is how many of the truly High hours the model catches. The baseline's perfect recall shows why accuracy and precision matter too: calling every hour busy catches every rush but is useless in practice. Both trained models beat the baseline by a wide margin, and Random Forest is best on every metric.

F.3 What the errors mean

For a bike operator, a **false positive** (31 hours) means trucks move bikes to stations that turn out not to need them, which wastes fuel and staff time. A **false negative** (47 hours) means a rush arrives at stations that weren't restocked, leaving commuters with no bikes. False negatives are the more costly mistake, so recall on the High class (94.6%) is the number operators should care about most.

F.4 Which features mattered

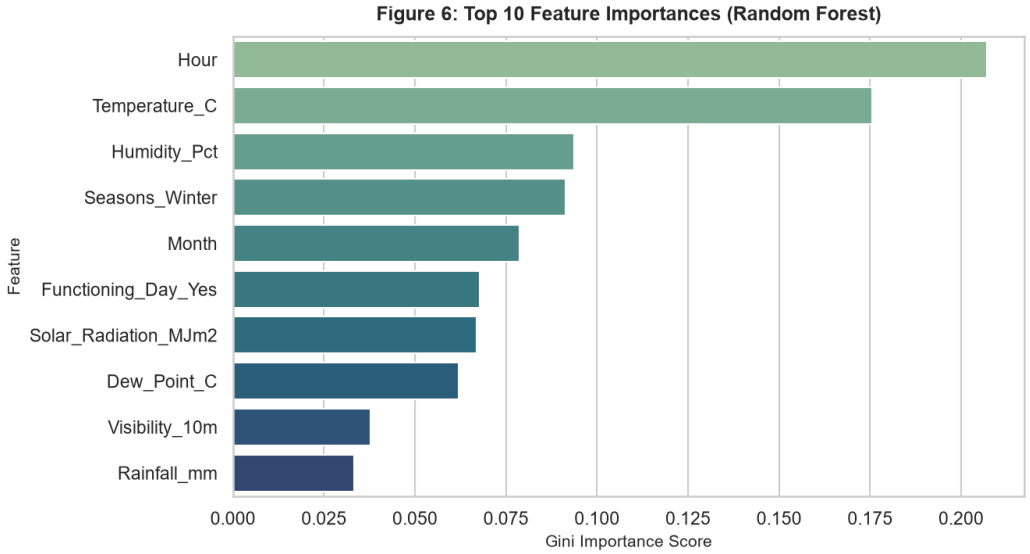


Figure 6. Top 10 feature importances from Random Forest (notebook: Feature importance cell)

The model relied most on Hour (20.7%), Temperature (17.6%), Humidity (9.4%) and the Winter season flag (9.1%), followed by Month, Functioning_Day and Solar Radiation. This matches the EDA: time of day and weather drive demand.

F.5 Error analysis

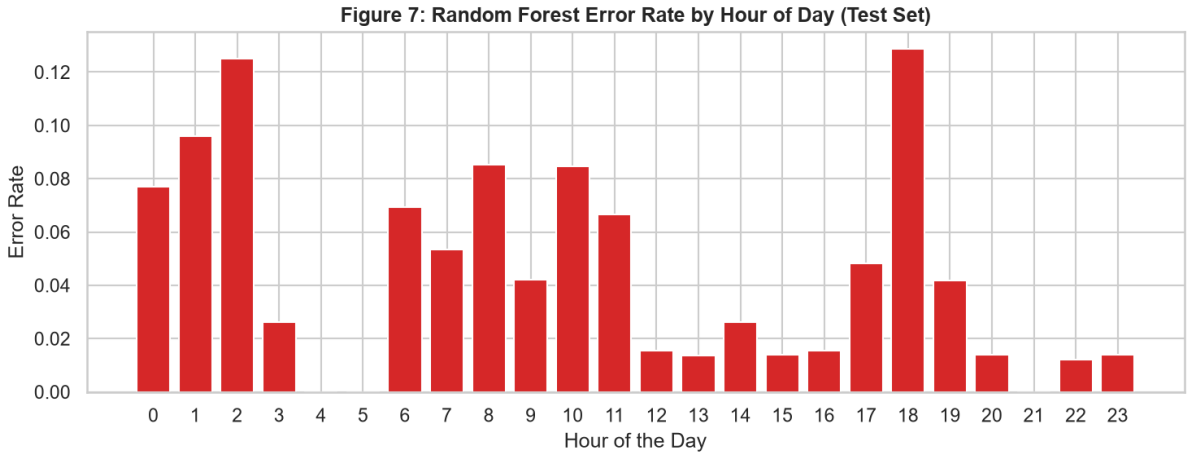


Figure 7. Random Forest error rate by hour of day (notebook: Error analysis cell)

We looked at the 78 hours Random Forest got wrong.

- **Most errors are borderline hours.** 54 of the 78 errors were hours with rentals within 150 of the 505 cutoff. Wrong predictions sat a median of 97 rentals from the cutoff; correct ones sat 378 away. The model rarely mislabels an hour that is clearly busy or clearly quiet.

- **The hardest hours are 6 PM (12.9% wrong) and 1 to 2 AM (about 10 to 12%)**, because demand at those hours sits close to 505 in some seasons. For example, the winter 6 PM average is 438.
- **Errors are spread fairly evenly across seasons**, from 3.5% in autumn to 5.1% in summer and winter.

F.6 Limitations

1. **The random split flatters the model.** Because the 80-20 split is random, the model can train on 7 AM and be tested on 8 AM of the same day, which are very similar. When we instead trained on the first 80% of the year and tested on the last 20% (Sep 19 to Nov 30), Random Forest dropped from 95.6% to 85.1%, while logistic regression stayed at about 86% (notebook: Time-split check cell). Real forecasting would look more like the second case.
2. **City-wide totals only.** The data counts rentals across all of Seoul, so the model can't say which station needs bikes.
3. **Old data.** The data covers 2017 to 2018, before the COVID-19 pandemic changed commuting habits.
4. **Missing causes.** Events, transit strikes and festivals affect demand but aren't in the data.
5. **Two classes only.** An hour with 510 rentals and one with 3,000 are both "High", which hides a big practical difference.

F.7 Conclusion

Weather, time of day and the calendar are enough to predict whether an hour in Seoul will have high or low bike demand. Random Forest performed best, with 95.6% accuracy and 94.6% recall on busy hours, compared with 86.9% for logistic regression and 49.5% for the baseline. Its remaining errors are mostly borderline hours near the cutoff. Its advantage shrinks when predicting future months, which is the most important point to keep in mind before using it in practice.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Sathishkumar V E, & Cho, Y. (2020). A rule-based model for Seoul Bike sharing demand prediction using weather data. *European Journal of Remote Sensing*. https://doi.org/10.1080/22797254.2020.1725789

Seoul Bike Sharing Demand [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C5F62R